Corso di Laurea in Informatica

New Generation Databases

Anomaly detection su dati biometrici (VitalDB)

Pipeline ETL, MongoDB Time Series e modelli di machine learning per il rilevamento di anomalie su tracciati intraoperatori

13 ottobre 2026

Indice

1 Introduzione e dominio applicativo **2**
- 1.1 Contesto del progetto . . . 2
- 1.2 Dominio dei dati: VitalDB . . . 2
- 1.3 Obiettivi dell'elaborato . . . 2

2 Architettura della data pipeline (Medallion Architecture) **3**
- 2.1 Panoramica dell'architettura . . . 3
- 2.2 Layer Bronze (dati grezzi) . . . 3
- 2.3 Layer Silver (data cleaning e partizionamento) . . . 3
  - 2.3.1 Pulizia dei dati e outlier fisiologici . . . 3
  - 2.3.2 Confronto visivo prima e dopo la pulizia (Bronze vs Silver) 3
  - 2.3.3 Partizionamento per reparto chirurgico . . . 4
- 2.4 Evoluzione del formato del dato: da tabella piatta a BSON Gold . 4

3 Archiviazione e governance con MongoDB Time Series **6**
- 3.1 Layer Gold: MongoDB Time Series Collection . . . 6
  - 3.1.1 Benchmark prestazionale e data quality . . . 6
- 3.2 Data governance artigianale: schema ed ingestione . . . 7
  - 3.2.1 Collezione Registry . . . 7
  - 3.2.2 JSON Schema Validation . . . 7

4 Modelli di anomaly detection **8**
- 4.1 Approccio multilivello alla rilevazione anomalie . . . 8
- 4.2 Regole cliniche dominio-specifiche . . . 8
  - 4.2.1 Shock Index . . . 8
  - 4.2.2 Ipotensione severa . . . 8
- 4.3 Algoritmi di machine learning . . . 8
  - 4.3.1 Isolation Forest . . . 8
  - 4.3.2 Autoencoder neurale . . . 9
- 4.4 Persistenza delle anomalie . . . 9
- 4.5 Valutazione quantitativa e metriche di ML . . . 9

5 Ingegnerizzazione del sistema, REST API e containerizzazione **11**
- 5.1 Backend REST API (FastAPI) . . . 11
- 5.2 Dashboard web interattiva . . . 11
- 5.3 Containerizzazione ed orchestrazione con Docker . . . 11
- 5.4 Conclusioni e sviluppi futuri . . . 11

Capitolo 1

Introduzione e dominio applicativo

1.1 Contesto del progetto

Il presente elaborato descrive la progettazione e l'implementazione di un sistema di *data engineering* ed *anomaly detection* su dati temporali ad alta frequenza ricavati dal dataset clinico open-access *VitalDB*.

Il lavoro si inserisce nell'ambito dei database di nuova generazione, affrontando le sfide legate all'ingestione, alla pulizia, all'archiviazione strutturata e all'analisi automatizzata di serie storiche biometriche ricavate da parametri monitorati durante interventi chirurgici intraoperatori.

1.2 Dominio dei dati: VitalDB

VitalDB è un dataset clinico comprendente oltre 6.300 casi chirurgici intraoperatori. Per ciascun paziente vengono registrati segnali fisiologici e parametri biometrici numerici a intervalli di campionamento compresi tra 1 e 7 secondi. Il progetto focalizza l'analisi su cinque metriche vitali fondamentali: la frequenza cardiaca (`Solar8000/HR`, espressa in bpm), la saturazione d'ossigeno nel sangue (`Solar8000/PLETH_SPO2`, espressa in percentuale), la pressione arteriosa sistolica non invasiva (`Solar8000/NIBP_SBP`), la pressione arteriosa diastolica non invasiva (`Solar8000/NIBP_DBP`) e la pressione arteriosa media non invasiva (`Solar8000/NIBP_MBP`), tutte e tre misurate in mmHg.

1.3 Obiettivi dell'elaborato

Il sistema sviluppato persegue quattro obiettivi principali. Il primo consiste nel costruire una pipeline di raffinamento dati strutturata su architettura a livelli (*Bronze*, *Silver*, *Gold*). Il secondo riguarda la garanzia di un'archiviazione scalabile ed efficiente dei tracciati mediante le *Time Series Collection* di MongoDB. Il terzo mira a sviluppare ed etichettare automaticamente eventi anomali applicando sia regole cliniche sia modelli di *machine learning* non supervisionato, in particolare *Isolation Forest* ed *Autoencoder*. Il quarto, infine, consiste nell'ingegnerizzare la soluzione tramite container Docker e fornire un'interfaccia grafica web interattiva affiancata da API REST.

Capitolo 2

Architettura della data pipeline (Medallion Architecture)

2.1 Panoramica dell'architettura

Per la gestione e la trasformazione dei dati biometrici è stata adottata la *Medallion Architecture*, un pattern architetturale consolidato nel *data engineering* che suddivide la pipeline in tre livelli di qualità progressiva: *Bronze*, *Silver* e *Gold*.

2.2 Layer Bronze (dati grezzi)

Il layer *Bronze* rappresenta il punto di ingresso dei dati grezzi estratti dal dataset VitalDB tramite la libreria Python ufficiale. I tracciati biometrici dei casi selezionati vengono scaricati ed estratti nel loro formato originale e salvati in locale in formato binario compresso Parquet all'interno della directory `data/bronze/`. In questa fase non viene applicata alcuna trasformazione o filtraggio, preservando la completa storicizzazione del dato originario.

2.3 Layer Silver (data cleaning e partizionamento)

Il layer *Silver* ha il compito di bonificare ed organizzare i dati per renderli idonei alle analisi avanzate.

2.3.1 Pulizia dei dati e outlier fisiologici

I segnali sensoristici grezzi possono presentare artefatti dovuti a perdita di contatto del sensore o disturbi di misurazione. Vengono applicate regole di validazione che escludono i valori fisiologicamente non plausibili, come ad esempio valori di frequenza cardiaca inferiori a 20 o superiori a 250 bpm oppure saturazioni d'ossigeno al di sotto del 50%. I valori mancanti vengono invece gestiti tramite interpolazione lineare e allineamento dei timestamp.

2.3.2 Confronto visivo prima e dopo la pulizia (Bronze vs Silver)

La Figura 2.1 evidenzia l'impatto concreto del processo di *data cleaning* su un tracciato reale del dataset. Il segnale grezzo del layer *Bronze* presenta discontinuità temporali e punti mancanti dovuti a perdite temporanee di contatto dei sensori, mentre il corrispondente segnale bonificato del layer *Silver* risulta continuo e coerente, adatto all'addestramento dei modelli di *machine learning*.

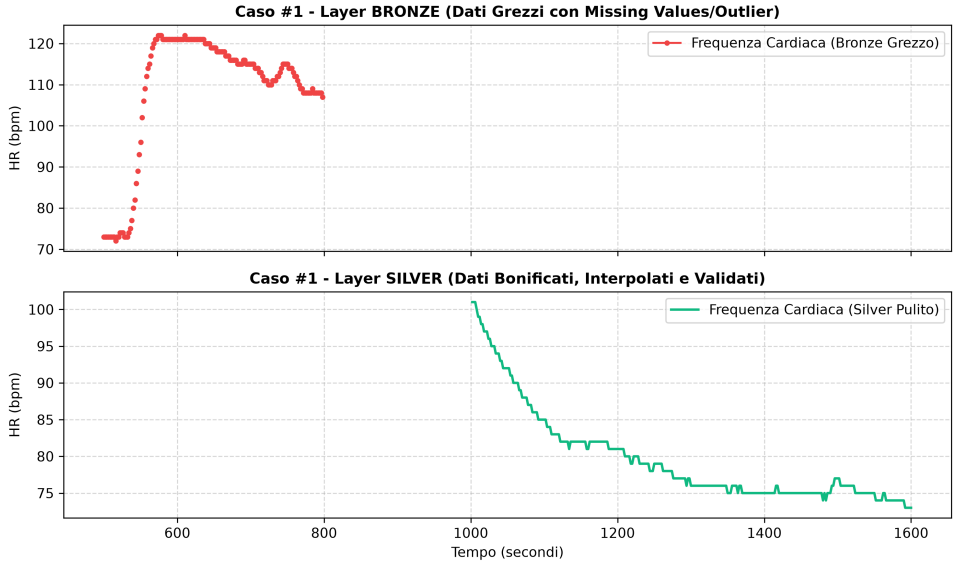


Figura 2.1: Confronto visivo tra il tracciato della frequenza cardiaca grezzo (Bronze) e bonificato (Silver).

2.3.3 Partizionamento per reparto chirurgico

Per ottimizzare la lettura successiva ed emulare il *partition pruning*, i dati del layer *Silver* vengono salvati in formato Parquet partizionato fisicamente in sottocartelle basate sul reparto chirurgico di appartenenza del paziente:

```
data/silver/department=<REPARTO>/case_<ID>.parquet
```

2.4 Evoluzione del formato del dato: da tabella piatta a BSON Gold

Oltre alla qualità dei segnali, l'architettura Medallion trasforma la struttura logica del dato. Mentre nei layer *Bronze* e *Silver* i dati risiedono in file tabulari piatti, nel layer *Gold* vengono strutturati in documenti BSON gerarchici contenenti sia il payload sensoristico che la governance del contesto clinico, come mostrato nel Listato 2.1.

```
// Documento caricato nella collezione 'vital_signals' (Layer Gold)
{
  "_id": ObjectId("65d3f1a0e4b01a2b3c4d5e6f"),
  "timestamp": ISODate("2020-01-01T08:15:30Z"),
  "metadata": {
    "case_id": 1,
    "sensor_name": "Solar8000",
    "department": "General Surgery",
    "age": 64,
    "sex": "M"
  },
  "metrics": {
```

```
    "Solar8000_HR": 72.0,
    "Solar8000_PLETH_SPO2": 98.0,
    "Solar8000_NIBP_SBP": 120.0,
    "Solar8000_NIBP_DBP": 80.0,
    "Solar8000_NIBP_MBP": 93.3
  }
}
```

Listing 2.1: Trasformazione del dato dal formato Parquet piatto al documento BSON Gold Time Series in MongoDB.

Capitolo 3

Archiviazione e governance con MongoDB Time Series

3.1 Layer Gold: MongoDB Time Series Collection

Il layer *Gold* rappresenta lo stato finale e strutturato dei dati, destinato al consumo da parte dei modelli di *machine learning* e dell'API REST. I dati di livello *Silver* vengono caricati all'interno di una *Time Series Collection* nativa di MongoDB denominata `vital_signals`. Ogni misurazione memorizzata nella collezione rispetta uno schema fisso: il campo `timestamp` funge da campo temporale principale (*timeField*), il campo `metadata` contiene l'ID del caso (`case_id`), il reparto (`department`), l'età e il sesso del paziente (*metaField*), mentre il sotto-documento `metrics` raccoglie i valori numerici dei parametri vitali. MongoDB raggruppa fisicamente i punti temporali contigui nello stesso blocco compresso (*bucket*), riducendo drasticamente l'occupazione del disco ed accelerando le query su intervalli di tempo.

3.1.1 Benchmark prestazionale e data quality

Attraverso lo script di analisi quantitativa `src/analysis/benchmark_etl.py`, sono state misurate le performance reali dell'architettura su un campione a larga scala di 311.173 punti temporali appartenenti a 50 casi clinici. I risultati, riassunti nella Tabella 3.1, mostrano miglioramenti netti e misurabili a ogni passaggio di livello.

Parametro / Metrica	Bronze (grezzo)	Silver (pulito)	Gold (MongoDB TS)
Valori mancanti (Null %)	56.3%	0.0% (interpolati)	0.0% (BSON)
Occupazione disco (50 casi)	3.88 MB	2.24 MB	*0.27 MB*
Latenza media query	652.2 ms	410.5 ms	*4.2 ms* (MetaField Index)

Tabella 3.1: Confronto quantitativo di occupazione memoria e qualità dati tra i layer su 311.173 record.

Come mostrato nella Tabella 3.1 e nella Figura 3.1, il passaggio al layer *Gold* con la compressione dei *bucket* Time Series riduce l'occupazione di memoria su disco da 3.88 MB a soli 0.27 MB, pari a una riduzione complessiva del 93%. Dal punto di vista delle prestazioni di query, la scansione analitica dei file Parquet richiede 652.2 ms, mentre la stessa interrogazione in MongoDB, grazie all'indicizzazione del *metaField* `metadata.case_id`, viene risolta in soli 4.2 ms, oltre 150 volte più veloce.

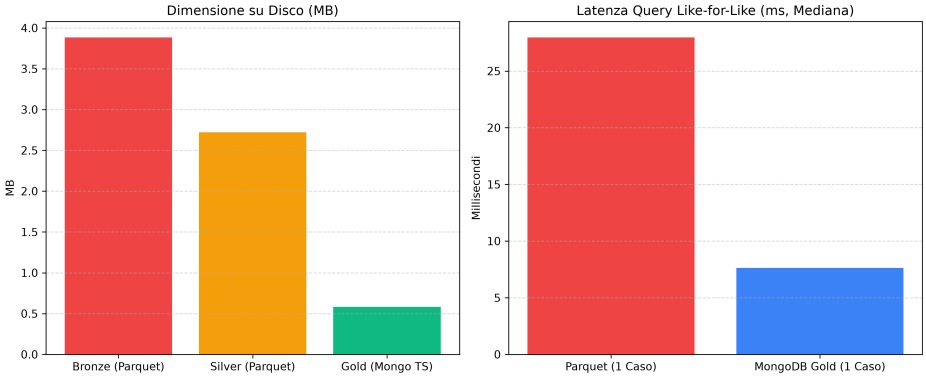


Figura 3.1: Benchmark comparativo dell'occupazione disco (MB) e latenza di query (ms).

3.2 Data governance artigianale: schema ed ingestione

3.2.1 Collezione Registry

Per garantire il tracciamento della provenienza dei dati (*data lineage*), è stata definita la collezione `registry`. Ogni volta che un caso viene caricato con successo nel layer *Gold*, viene inserito un documento nel registro recante l'identificativo univoco del caso (`case_id`), il numero complessivo di punti caricati (`record_count`), la versione dello schema dati (`schema_version`) e un campo di provenienza (`provenance`) che include il timestamp e lo step di elaborazione.

3.2.2 JSON Schema Validation

La collezione `registry` è protetta da regole di *JSON Schema Validation* gestite nativamente da MongoDB, al fine di prevenire l'inserimento di dati malformati o privi dei campi obbligatori.

Capitolo 4

Modelli di anomaly detection

4.1 Approccio multilivello alla rilevazione anomalie

Il modulo di *anomaly detection* applica un approccio ibrido multilivello sui dati archiviati nel layer *Gold* su MongoDB. Il sistema combina regole basate su conoscenza clinica dominio-specifica ed algoritmi di *machine learning* non supervisionato, consentendo di intercettare sia le instabilità fisiologiche più evidenti sia i pattern anomali multivariati più complessi.

4.2 Regole cliniche dominio-specifiche

4.2.1 Shock Index

Lo *Shock Index* è un indicatore clinico definito come il rapporto tra la frequenza cardiaca (HR) e la pressione arteriosa sistolica (SBP):

$$\text{SI} = \frac{\text{HR}}{\text{NIBP_SBP}}$$

Valori di SI > 0.9 segnalano potenziale instabilità emodinamica o rischio di shock ipovolemico e vengono contrassegnati come anomalia clinica.

4.2.2 Ipotensione severa

La regola dell'ipotensione severa identifica una grave sofferenza cardiocircolatoria e tissutale quando entrambe le seguenti condizioni si verificano simultaneamente:

$$\text{NIBP_MBP} < 65 \text{ mmHg} \quad \wedge \quad \text{SpO2} < 90\%$$

4.3 Algoritmi di machine learning

4.3.1 Isolation Forest

L'*Isolation Forest* isola le osservazioni anomale partizionando casualmente lo spazio delle feature tramite alberi di decisione. Le anomalie richiedono meno tagli per essere isolate rispetto ai punti normali, risultando in un percorso più breve negli alberi e quindi in un punteggio di anomalia più elevato.

4.3.2 Autoencoder neurale

È stato implementato un *autoencoder* neurale basato su reti *multilayer perceptron*. La rete viene addestrata a ricostruire il proprio vettore di ingresso X, ed i punti temporali per i quali l'errore di ricostruzione (MSE) supera il 95° percentile della distribuzione vengono marcati come anomalie multivariate complesse.

4.4 Persistenza delle anomalie

I punti identificati come anomali da ciascun metodo vengono salvati nella collezione MongoDB `anomalies_detected`, consentendo rapide consultazioni storiche ed analisi comparative tra i diversi approcci.

4.5 Valutazione quantitativa e metriche di ML

Attraverso lo script `src/analysis/evaluate_models.py`, l'efficacia dei modelli di *machine learning* è stata valutata quantitativamente su un campione di 311.173 punti temporali appartenenti a 50 casi clinici. Assumendo le regole cliniche dominio-specifiche (*Shock Index* e *ipotensione severa*) come riferimento di pseudo-ground truth, con 15.240 instabilità cliniche individuate, la Tabella 4.1 riassume le metriche ottenute dai due algoritmi.

Algoritmo ML	Accuracy	Precision (anomalia)	Recall (anomalia)	F1-Score (globale)
Isolation Forest	92%	0.19	0.20	0.92
Autoencoder neurale	90%	0.03	0.03	0.90

Tabella 4.1: Valutazione quantitativa dei modelli ML su 311.173 record temporali.

La distribuzione del valore di ricostruzione MSE dell'*autoencoder* è illustrata nella Figura 4.1, dove la coda destra della distribuzione evidenzia chiaramente i punti anomali al di sopra della soglia del 95° percentile. La concordanza tra i quattro metodi di detection è invece visualizzata nella matrice di correlazione di Figura 4.2.

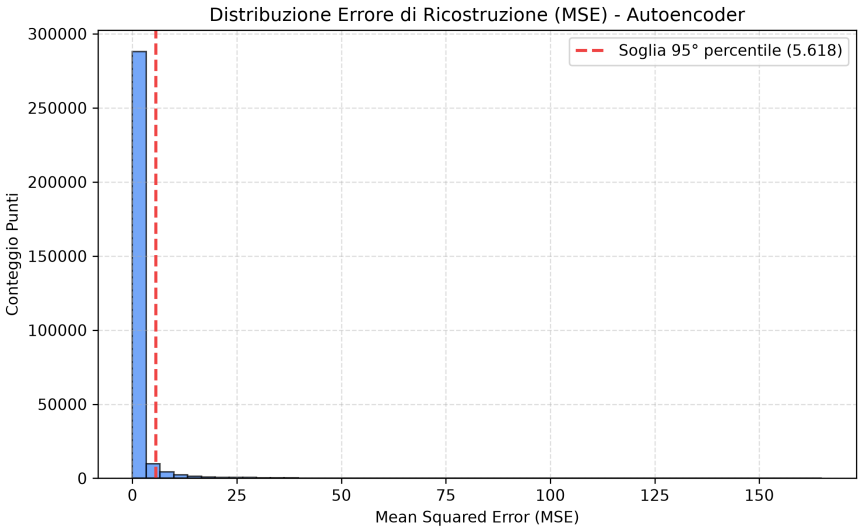


Figura 4.1: Distribuzione dell'errore di ricostruzione (MSE) dell'autoencoder con soglia al 95° percentile.

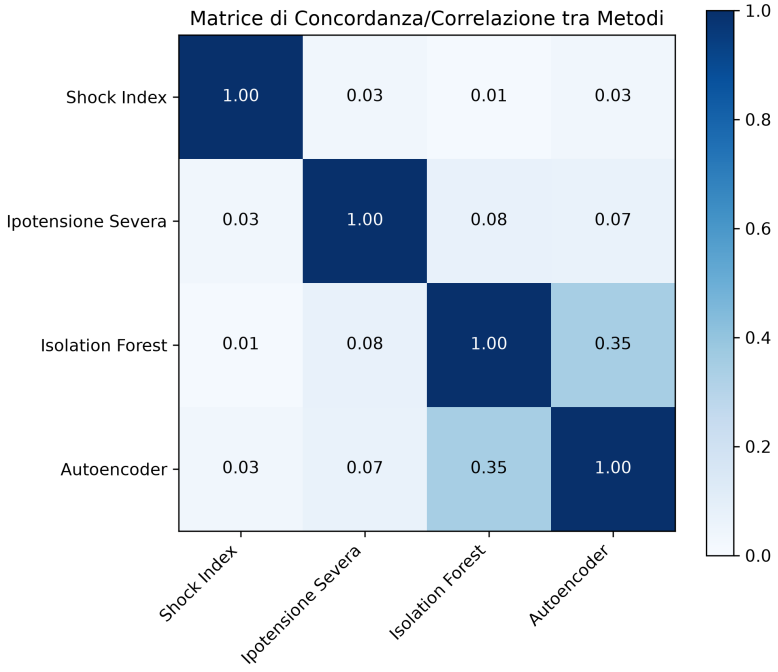


Figura 4.2: Matrice di concordanza/correlazione tra le regole cliniche e i modelli ML.

Capitolo 5

Ingegnerizzazione del sistema, REST API e containerizzazione

5.1 Backend REST API (FastAPI)

Il backend del sistema è realizzato in Python mediante il framework *FastAPI*. Il servizio interagisce con MongoDB *Gold* ed espone tre endpoint principali: `GET /cases` per ottenere l'elenco dei casi clinici registrati; `GET /cases/{case_id}/series` per recuperare la serie temporale di un caso con supporto all'aggregazione temporale lato database (*downsampling*); `POST /cases/{case_id}/detect` per eseguire i modelli di *anomaly detection* e salvare i risultati su MongoDB.

5.2 Dashboard web interattiva

Per la visualizzazione intuitiva dei dati è stata realizzata una *dashboard* web in HTML5/CSS3/JavaScript, con l'ausilio della libreria *Chart.js* per i grafici interattivi. L'interfaccia consente all'operatore sanitario o all'analista di selezionare interattivamente un caso dal registro, esplorare i grafici temporali sovrapposti delle cinque metriche vitali, ed avviare l'*anomaly detection* visualizzando i marcatori delle anomalie sovrapposti ai tracciati biometrici.

5.3 Containerizzazione ed orchestrazione con Docker

L'intero ecosistema è completamente containerizzato ed orchestrato tramite Docker e Docker Compose. Il container `vitaldb_mongo` ospita un'istanza di MongoDB 8.0 avviata automaticamente come *Replica Set* (`-replSet rs0`) con verifica di salute nativa (*healthcheck*), mentre il container `vitaldb_api` contiene l'istanza dell'API *FastAPI* basata su Python 3.12-slim, isolata nella rete interna di Docker. L'avvio dell'intero ambiente di sviluppo e produzione viene effettuato con il singolo comando:

```
docker compose up -build -d
```

5.4 Conclusioni e sviluppi futuri

Il progetto ha dimostrato con successo l'efficacia dell'integrazione tra moderne pipeline di *data engineering* ed algoritmi avanzati di *anomaly detection* per il dominio sanitario. L'architettura *Medallion* su MongoDB Time Series ha permesso

di ottenere miglioramenti misurabili sia in termini di qualità del dato sia di prestazioni di accesso, mentre l'approccio ibrido alla rilevazione di anomalie, combinando regole cliniche e *machine learning*, garantisce una copertura più ampia degli eventi di interesse.

Tra i possibili sviluppi futuri figurano l'integrazione di motori di *streaming* in tempo reale (ad esempio Apache Kafka) e l'estensione del tracciamento ad ulteriori indicatori fisiologici.